

Weekly Mini Project 01 — Mystery Room

1. Project title

Mystery Room — an interactive, browser-based mystery/escape experience, built by your team from scratch.

2. Why we're building it

Every session so far has taught you real concepts using one project: **B4F Hub**. That's deliberate — a shared project lets a whole cohort learn the same idea from the same real code. But a shared project can also make it easy to forget that these concepts are *yours* now, not "the opportunities-app's" — they work anywhere.

This project asks you to prove that. You will design and build a **completely different product** — not another list of opportunities or community posts with the names changed — using the same technical tools Sessions 01–06 already gave you: React, React Router, component state, Context/Redux where it's genuinely needed, and a real Express backend with routes, controllers, and validation.

You will also **present it in English**, as a team, to practice explaining a real product you built — not just reciting code.

3. The idea, in plain English

Your team builds a small interactive mystery or escape-room experience that runs in the browser. A player opens your app, reads about a mystery, looks at clues, and submits an answer or decision. Your backend checks whether it's right and tells the frontend what happens next. Get enough stages right, and the mystery is solved — with a final reveal.

This is **not** a management dashboard, a CRUD admin panel, or "B4F Hub with different labels." It is an experience someone would actually want to click through and enjoy solving.

4. Example mystery concepts (inspiration only — none of these is required)

These are here to make the idea concrete, not to limit you. Invent your own theme; these are only examples of the *kind* of thing that works well:

- The Missing Laptop
- Escape from the Abandoned Hotel
- Who Stole the Exam Papers?
- The Secret Lab
- Lost in Damascus 2050
- The Hacker Who Disappeared
- A zombie survival mystery

- A treasure hunt
- A cybersecurity investigation
- A historical mystery

You do not have to pick from this list, and no two teams should end up with the same story. Surprise us.

5. Example user journey

TEXT

```
Home / Intro
  ↓
Choose or start a mystery
  ↓
Read the situation
  ↓
Inspect clues
  ↓
Submit an answer / decision
  ↓
Backend validates/processes it
  ↓
Correct → next step    /    Wrong → try again
  ↓
Complete the mystery
  ↓
Final result / reveal
```

This is the shape of the experience, not an exact file-by-file blueprint — how you tell your story inside this shape is up to you.

6. Mandatory requirements (overview)

Your project must:

- be a real, working React frontend talking to a real, working Express backend;
- use React Router with at least one dynamic route (the mystery itself is a natural fit);
- use a real backend API with genuine validation, real status codes, and real error handling;
- feel like an experience, not a data table;
- have an original theme, story, and wording your team invented.

The detailed technical contract is in sections 7–10 below.

7. Frontend requirements

- A real React frontend (the same stack you already know from B4F Hub: React + React Router; TypeScript is encouraged since you already know it, but not mandated if your team has a good reason).

- **At least 3 meaningful routes** (a home/landing page, a mystery/play screen, and at least one more — a "how to play" page, a results page, a mystery-selection page, etc. — your call).
- **At least 1 dynamic route** (e.g. a specific mystery, selected by id or slug in the URL).
- Real navigation using React Router (`Link / NavLink` , and `useNavigate` if useful).
- **At least one real, backend-driven screen** — a screen whose content genuinely comes from your API, not hardcoded in the component.
- All backend calls go through an API/helper layer (a module like `api.ts / api.js` with named functions), not `fetch` scattered directly inside components.
- A visible **loading state** while a request is in flight.
- A visible **error state** when a request fails, with a way to recover (retry, or at least not crash the page).
- A real, interactive answer/decision input the player actually uses to progress.
- A **final completion/result screen** that clearly tells the player the mystery is over and how it ends.

8. Backend requirements

- Node.js + Express (the same real stack from Sessions 01–06).
- Route/controller organization, the way Session 06 taught it: at least one real `express.Router()` , mounted in your entry file, and at least one controller module exporting the functions that actually handle requests.
- Shared, in-memory runtime state where your app genuinely needs it — following Session 06's own lesson: a clear, intentional owner of *current* state (not just re-imported seed data scattered across files, and not a raw duplicate array built independently in two places).
- **At minimum, these real endpoints, each with a genuine purpose in your story:**
 - one **GET** endpoint returning a collection or starting data (e.g. the list of available mysteries, or the initial state of your one mystery);
 - one **GET** `/:id` -style endpoint returning one specific thing by id;
 - one **POST** endpoint that reads `req.body` and does something meaningful with it (e.g. submitting an answer);
 - one **PATCH** endpoint that changes existing state (e.g. requesting a hint, updating progress).
- Real backend validation on anything the client sends — checked type, checked shape, checked value, not just trusted.
- Meaningful status codes — **at least one real 400** (invalid input) and **at least one real 404** (nothing found) in your actual, working API — not invented just to tick a box. A `409` (a genuine conflict with current state, e.g. "already solved") is a nice extra if your story has a natural place for one, but is not required.
- Independent testing — you must be able to prove every endpoint works with curl/Postman/Thunder Client, without the frontend involved at all.

9. API behavior expectations

Design your own API. Here is one *possible* shape, to make the requirements concrete — not the exact endpoints you must build:

HTTP

```
GET /api/mysteries
GET /api/mysteries/:id
GET /api/mysteries/:id/clues

POST /api/mysteries/:id/answers
PATCH /api/mysteries/:id/hint
```

Example request:

JSON

```
{ "answer": "0mar" }
```

Example response:

JSON

```
{ "correct": false, "message": "That does not match the evidence." }
```

Your own endpoints, field names, and response shapes should fit *your* story. A treasure-hunt app might have `/api/hunts/:id/dig` instead of `/api/mysteries/:id/answers` — same underlying concepts, different words, because it's a different product.

10. Validation/error expectations

- A request with a missing or wrong-typed required field returns `400` with a clear error message — never a silent success, never a server crash.
- A request for something that doesn't exist (a bad id) returns `404`, not a `200` with empty/null data.
- Your frontend must handle both cases without crashing — a failed request shows something sensible to the player, not a blank screen or an unhandled promise rejection.

11. Minimum UI/UX requirements

- The experience should read clearly from top to bottom — a new player should understand what to do without you explaining it out loud.
- Use real content: your own story, your own clue text, your own wording. No lorem ipsum, no placeholder text left in the final submission.
- A visual identity of your own choosing (colors, a name, a logo/emoji, a tone) — it does not need to be professionally designed, but it should feel intentional, not default-browser-styled.

12. Creativity requirements

- An original project name.
- An original theme/story your team invented.
- Original clue content and wording — not copied from the inspiration list or from another team.
- A coherent final reveal that actually resolves the story you set up.

Originality of theme, story, and experience is graded. A technically correct project that is just "the example API with different field names and no real story" will not score as well as one with a genuine, coherent idea — even a simple one.

13. What teams may choose freely

- Your theme, story, characters, and setting.
- Your exact API endpoint names and payload shapes (as long as the required *kinds* of endpoints exist and behave correctly).
- How many mysteries/stages/clues you build (one solid mystery with a few real stages is worth more than five shallow ones).
- Whether you use TypeScript or plain JavaScript on the frontend.
- Whether you use Context, Redux Toolkit, or neither — see section "Choosing your state management" below.
- Whether you use `localStorage` for anything — only if your team has a genuine reason (e.g. remembering progress across a reload).
- Your visual design and branding.

Choosing your state management (read this before reaching for Redux)

You've been taught local `useState`, Context, and Redux Toolkit. **This project does not require any specific one of them.** Reason about ownership, the same way Session 06 taught you to reason about backend state ownership:

TEXT

Which mystery is currently open?	→ the URL (React Router), not a slice
One input field's current text?	→ local <code>useState</code> in that component
Data that only one screen needs?	→ fetch it there, keep it local
Something genuinely shared across several distant components at once?	→ Context, or Redux Toolkit if the sharing is real and non-trivial
Progress that should survive a reload?	→ <code>localStorage</code> , only if your team decides it's actually worth it
The real, current state of the mystery (solved? which stage? which hints?)	→ your backend — the frontend should ask it, not invent its own copy

Using Redux "because we learned it" for something that's really just one component's local state is not a stronger submission — it's a sign the ownership reasoning wasn't done. The same is true in reverse: don't avoid Context/Redux out of fear if you genuinely do have state shared across several distant parts of your app.

14. What is forbidden / out of scope

Do not use, and do not need, any of the following — they are either not taught yet or deliberately unnecessary for this project:

- MongoDB, PostgreSQL, Prisma, Mongoose, or any database/ORM;

- authentication, login/register, JWT, sessions, cookie-based auth, password hashing;
- NestJS, Next.js;
- GraphQL, WebSockets, Redis;
- Docker or deployment of any kind;
- Redux `createAsyncThunk`, RTK Query, or React Query;
- Zod, Joi, `express-validator`, or any other validation library — plain `if / typeof` checks are exactly what Sessions 05–06 taught, and exactly what's expected here;
- complex custom error-handling middleware, service/repository layers, or dependency injection.

If your design starts needing one of these to work, that's a sign the design has grown beyond this project's scope — simplify the idea, don't reach for a bigger tool.

15. Team collaboration expectations

- Every team member commits real, visible work — no single person should author the whole project alone.
- Everyone should be able to explain the whole project, not only "their part," by presentation day.
- Agree on your API shape and your data model together, early, before splitting up work — this avoids the frontend and backend disagreeing about what a response looks like.

16. Git expectations

- Commit regularly, with messages that describe what actually changed — not one giant "final commit."
- Work on your own branches / a workflow your team agrees on; avoid one person pushing everything at the end.
- Everyone's contribution should be visible in the repository's history.
- Keep the backend and frontend both easy to run with clear install/start instructions in your own README.

This does not need a complicated Git process — a shared repository with regular, real commits from every member is enough.

17. Definition of done

Your project is done when:

- a fresh install (`npm install` in both `client/` and `server/`) and start (`npm run dev` / `npm start`) actually works, from a clean checkout;
- a player can go from the home screen to a solved mystery using only the UI, with no console errors breaking the flow;
- every required endpoint (section 8) is independently testable and behaves correctly, including its failure cases;
- the story, clues, and wording are your team's own; and
- every team member can explain the whole project and has a real speaking role in the presentation.

18. Presentation requirements

You will present your project **in English**, as a team, during a soft-skills session. See `PRESENTATION-GUIDE-EN.md` for the full structure, timing, and sentence starters. In short: this is not a code walkthrough — it's you telling us about a product you built, showing it working, and reflecting honestly on what was hard and what you'd improve.

19. English presentation structure (summary)

Roughly 5–7 minutes per team:

1. Team + project introduction (~1 min)
2. Story / problem / user goal (~1 min)
3. Live demo (~2 min)
4. Technical architecture (~1–2 min)
5. Challenge faced + what you'd improve with more time (~1 min)

Full detail, including sentence starters, is in `PRESENTATION-GUIDE-EN.md`.

20. Presentation timing recommendation

5–7 minutes per team, plus a short buffer for questions. Rehearse once as a full team before presentation day — even a rough rehearsal makes a real difference.

21. Demo checklist

Before presenting, confirm:

- ☐ Both servers start cleanly from a fresh clone.
- ☐ The home screen loads with no visible errors.
- ☐ At least one full mystery can be solved live, start to finish.
- ☐ A wrong-answer case is shown at least once (so we see real validation, not just the happy path).
- ☐ The final result/reveal screen actually appears and makes sense.
- ☐ Every team member knows which part they're presenting.

22. Grading/evaluation rubric

See `../instructor/GRADING-RUBRIC.md` for the instructor's full weighting. In summary, you are evaluated on: the product genuinely working, correct use of the concepts taught, real frontend/backend integration, real backend validation, reasonable code organization, visible team contribution, the originality of your product idea, and your English presentation/demo. **Visual polish is not a dominant factor** — a technically solid project with a simple but coherent UI scores well.

23. Optional ideas (using only concepts you already know)

None of these are required. They're here only if your team finishes the core experience with time to spare:

- a second, shorter mystery in the same app;
- a simple score/timer shown on the result screen;
- a hint system with a visible "hints remaining" count (the instructor's own example does this — feel free to build your own version);
- remembering completed mysteries in `localStorage` so a badge survives a reload;
- a small "share your result" text block a player could copy.

Do not add any of these at the cost of the mandatory requirements above.